

ANDROID STUDIO

HARTZAILEAK ETA LUZAPENAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Funtzio independenteak eta objektu-funtzioak
2. Hartzaileentzako sarrera
3. Hedapen-funtzioetarako sarrera
4. Hedaduraren propietateak

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Hona hemen kate baten hasieran eta bukaeran koma bakunak jartzen dituen funtzio oso sinple bat:

```
fun singleQuoted(original: String) = "'$original'"
```

Gogoratuko duzunez, funtzio hau erraz deitu daiteke, honela:

```
val title = "The Robots from Planet X3"  
val quotedTitle = singleQuoted(title)  
  
println(quotedTitle) // 'The Robots from Planet X3'
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Ondoren, objektuek ere funtzioak izan ditzaketela ikasi genuen. Adibidez, string motako objektuek `uppercase()` izeneko funtzio bat dute, zeinak kate bera itzultzen duen, baina hizki guztiak maiuskulaz. Honela dei diezaiokezu:

```
val title = "The Robots from Planet X3"  
val loudTitle = title.uppercase()  
  
println(loudTitle) // THE ROBOTS FROM PLANET X3
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Objektu batean dagoen funtzio bati deitzen zaionean, objektuaren izena eta puntu bat jartzen dira funtzioaren deia baino lehen. Horregatik, deialdi mota horri **puntu-notazioa** deritzo.

Horrela, bi funtzio-kategoria ditugu:

- Objektu batetik aparte modu independentean funtzionatzen duten funtzioak.
- Objektu batean deitzen diren funtzioak.

FUNTZIO INDEPENDENTAK ETA OBJEKTU-FUNTZIOAK

Calling a standalone
function

`singleQuoted(title)`



No object name or dot

Calling a function on
an object

`title.uppercase()`



Object name and dot

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Hala ere, gauzak korapilatu egiten dira bi funtzio mota hauek toki berean konbinatzen direnean. Begiratu kode honi: `singleQuoted()` funtzio independente bati eta puntu-notazioan dauden bi funtziori (`removePrefix()` eta `uppercase()`) dei egiten die:

```
singleQuoted(title.removePrefix("The ")).uppercase()
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Asmatu dezakezu funtzio hauei zein ordenatan deituko zaien?

1. Lehenengo, `removePrefix()` deitzen da.
2. Ondoren, deialdi horren emaitza `singleQuoted()` funtzioaren argumentu bezala erabiliko da.
3. Azkenik, `uppercase()` `singleQuoted()`-ek itzultzen duen String objektuan deituko da.

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Ikusmen-mailan, gure buruak adierazpen hau prozesatu behar du alde batetik bestera, erdi aldean hasiz, gero ezkerretara mugituz eta azkenik eskuinetara. Zaila litzateke horrelako liburu bat irakurtzea!

```
singleQuoted(title.removePrefix("The ")).uppercase()
```



FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Ikusmen-mailan, gure buruak adierazpen hau prozesatu behar du alde batetik bestera, erdi aldean hasiz, gero ezkerretara mugituz eta azkenik eskuinetara. Zaila litzateke horrelako liburu bat irakurtzea!



fox jumps (the quick brown) over the lazy dog

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Kodea errazagoa litzateke irakurtzeko eta ulertzeko, hiru funtzio-deialdiak modu berean funtzionatuko balira, eta norabide bakarrean irakurtzea ahalbidetuko baligute. Adibidez, puntu-notazioa erabil genezake `singleQuoted()` funtzioari deitzeko (nola egiten dugun `removePrefix()` eta `uppercase()`-rekin), jarraitzeko askoz ere errazagoa izango litzateke. Horrela ikusiko litzateke:

```
val newTitle = title.removePrefix("The ").singleQuoted().uppercase()
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

`singleQuoted()` `String` klasearen parte ez denez, kode honek oraindik ez du funtzionatzen. Hala ere, irakurtzeko eta ulertzeko askoz errazagoa izango litzateke, funtzioei irakurtzen diren ordenan deitzen zaielako: ezkerretik eskuinera.

```
title.removePrefix("The ").singleQuoted().uppercase()
```

1 —————> 2 —————> 3

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Deialdi hauek bertikalki ere antola daitezke, lerro bakoitzean bat:

```
val newTitle = title  
    .removePrefix("The ")  
    .singleQuoted()  
    .uppercase()
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Berriro ere, naturala da goitik behera irakurtzea.

Konsistentzia eta irakurgarritasuna hobetzeaz gain, badira kasuak puntu-notazioa garatzaile bati espero bezala errazago iruditzen zaizkionak, Kotlin-en erabiltzen duenean. Ohituraz, funtzio batek objektu batean edo objektu batekin zerbait egiten badu, askotan espero da funtzio hori objektuan egotea.

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Gainera, IntelliJ edo Android Studio bezalako IDE bat erabiltzen ari bazara, "objektu batean" dauden funtzioak aurkitzeko errazagoak dira. **String** objektu bat baduzu eta jakin nahi baduzu zer funtzio dauden eskuragarri, puntu bat idatzi eta ikusiko duzu funtzioen zerrenda bat. Klase berriak esploratzeko modu bikaina da hau.

FUNTZIO INDEPENDENTIAK ETA OBJEKTU-FUNTZIOAK

title.

uppercase() for String in kotlin.text	String
uppercase(locale: Locale) for String in kotlin..	String
substring(range: IntRange) for String in kotlin.text	String
substring(startIndex: Int) for String in kotli..	String
substring(startIndex: Int, endIndex: Int) for ..	String
removePrefix(prefix: CharSequence) for String ..	String
startsWith(prefix: String, ignoreCase: Boolean..	Boolean
startsWith(prefix: String, startIndex: Int, i..	Boolean
replaceFirst(oldChar: Char, newChar: Char, ign..	String
replace(oldChar: Char, newChar: Char, ignoreCa..	String
replace(oldValue: String, newValue: String, in	String

```
public fun String.substring(  
    range: IntRange  
): String
```

Returns a substring specified by the given
[range](#) of indices.

[kotlin.text](#)
StringsKt.class

Gradle: org.jetbrains.kotlin:kotlin-stdlib:1.6.21
(kotlin-stdlib-1.6.21.jar)

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Beraz, gure helburua `singleQuoted()` aldatzea da, puntu batekin dei ahal izateko, honela:

```
val newTitle = title.singleQuoted()
```

Analizatu ditzagun funtzio independenteen eta objektu batean dei egiten diren funtzioen arteko antzekotasunak eta desberdintasunak.

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Bi funtzio-kategoria hauek (funtzio independenteak eta objektu batean deitzen diren funtzioak) uste baino gehiago dute komunean. Bai, funtzioari dei egiteko idatzi behar den kodearen forma (hau da, sintaxia) zertxobait ezberdina da kasu bakoitzean:

```
singleQuoted(title)
```

|

```
title.uppercase()
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Baina kontzeptuari dagokionez, benetan antzekoak dira:

- Biak kate batetik hasten dira.
- Biak jatorrizko katean oinarritutako kate berri bat itzultzen dute.

Ikuspuntu horretatik, bi funtzioek **String** argumentu bat hartuko balute bezala da. Desberdintasun bakarra argumentu hori non kokatzen den da, funtzioari deitzen zaionean.

```
singleQuoted(title) | title.uppercase()  
title argument     | title argument
```

FUNTZIO INDEPENDENTEAK ETA OBJEKTU-FUNTZIOAK

Puntu baten bidez funtzio bati deituz gero, puntuaren ezkerrean dagoen objektua funtzio deialdiaren **hartzaila** deritza. Hartzailak kapitulu honetako kontzeptu garrantzitsuak dira, baina etorkizuneko kontzeptuak ulertzeko ere baliagarriak dira, hala nola esparru-funtzioak eta lambda aurreratuagoak, beraz, goazen sakonago ikertzera!

KORRUTINEN OINARRIAK

Txakur ongi entrenatu batek zaunka egiten du agindu bat jasotzean. Zure txakurrari "hitz egin" esaten diozunean, agindu bat bidaltzen ari zara, eta bera da aginduaren hartzailea. Antzera, objektu batean funtzio bat deitzen duzunean, objektu hori da funtzio-deiaren **hartzailea**.



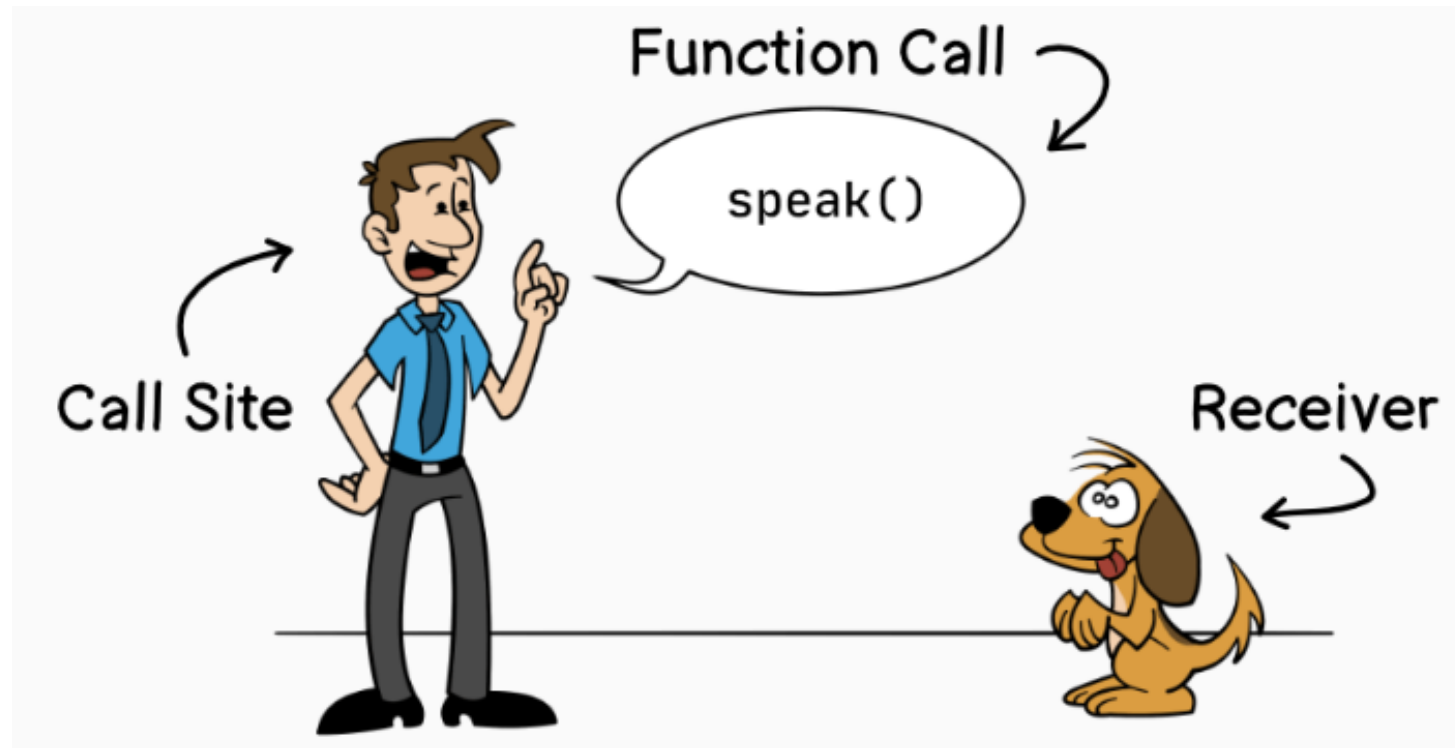
KORRUTINEN OINARRIAK

Txakur ongi entrenatu batek zaunka egiten du agindu bat jasotzean. Zure txakurrari "hitz egin" esaten diozunean, agindu bat bidaltzen ari zara, eta bera da aginduaren hartzailea..



KORRUTINEN OINARRIAK

Antzera, objektu batean funtzio bat deitzen duzunean, objektu hori da funtzio-deiaren **hartzailea**



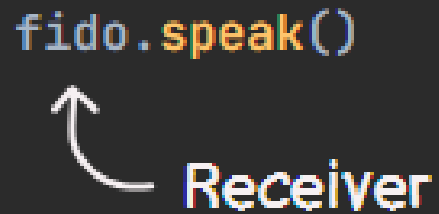
KORRUTINEN OINARRIAK

Hau zehatzago azalduko dugu kode apur batekin. Hemen da **Dog** izeneko klase sinple bat, txakurrari hitz egiteko eskatzen dion kode batekin jarraitzen duena.

```
class Dog {  
    fun speak() {  
        println("BARK!")  
    }  
}  
  
val fido = Dog()  
fido.speak()
```


KORRUTINEN OINARRIAK

Fido da **speak()** deitzen diozun txakurra, beraz, Fido da hartzailea.



```
fido.speak()  
    ↑  
Receiver
```

Erraza, ezta?

Batzuetan, ez duzu behar zure txakurrari hitz egiteko esatea. Batzuetan, berak aukeratuko du zaunka egitea (Egia esan, batzuetan ezin duzu zaunka egiteari uzteko konbentzitu... galdetu niri nola dakidan!)

KORRUTINEN OINARRIAK

Eguneratu dezagun **Dog** klasea Fido jolasten hasten den bakoitzean zaunka egin dezan.

```
class Dog {  
    fun speak() {  
        println("BARK!")  
    }  
    fun play() {  
        this.speak()  
    }  
}
```

KORRUTINEN OINARRIAK

Hemen, `play()` funtzioak `speak()` funtzioa deitzen du. `This` gako-hitza `play()` deitzen den objektu berari dagokio. Bestela esanda, `fido.play()` deitzen baduzu, `speak()` deituko zaio `fido` objektuan. `Speak()` funtzio-deiaren hartzailea `this` da.

Halaber, gogoratu dezakezu `this`. kendu dezakezula, beraz, hurrengo kodeak aurreko zerrendako kodeak bezala funtzionatzen du.

```
class Dog {  
    fun speak() {  
        println("BARK!")  
    }  
    fun play() {  
        speak()  
    }  
}
```

KORRUTINEN OINARRIAK

Orain, ez dago objektu-izenik ezta punturik **Speak()** aurretik; funtzioaren izena bakarrik dago. Honek esan nahi al du hemen ez dagoela hartzailerik?

```
class Dog {  
    fun speak() {  
        println("BARK!")  
    }  
    fun play() {  
        speak()  
    }  
}
```

Any receiver here?

KORRUTINEN OINARRIAK

Izatez, *hartzaile* bat dago hemen! Gogoratu: funtzio bat *objektu batean* deitzen den bakoitzean, objektu hori da hartzailea. **Dog** objektu batean **speak()** deitzen denez, objektu hori da hartzailea. **play()** funtzioaren barruan, **this.** gehi dezakezu **speak()** aurretik edo ezabatu dezakezu. Eraitza eta hartzailea berdinak dira bi kasuetan.

KORRUTINEN OINARRIAK

Beraz, **speak()** funtzioak hartzailea du hemen! Kodean ageriko ez den arren, isilik dago. Horregatik deitzen zaio **hartzaile isila**. Aldiz, goiko 10.8. Zerrendan dagoen **hartzaile agerikoarekin** alderatu. Hemen **speak()** deitzeko bi aukera erakusten dira: bat hartzaile isila eta beste bat hartzaile agerikoa erabiltzen duena.

KORRUTINEN OINARRIAK

```
class Dog {  
    fun speak() {  
        println("BARK!")  
    }  
    fun play() {  
        speak()  
    }  
}
```

(Implicit Receiver)

```
val fido = Dog()  
fido.speak()
```

Explicit Receiver

KORRUTINEN OINARRIAK

Hau guztia laburbildu dezakegu honela:

- Hartzailea zure deitzen ari zaren funtzioa duen objektua da.
- *Agerikoa* izan daiteke, puntu batekin funtzio bat deitzean ikusten den bezala, edo...
- *Isila* izan daiteke, klase bereko funtzio batek beste funtzio bat deitzen duenean bezala.

Hartzaileei buruzko ezagutza hau erabil dezakegu gure jatorrizko helburura itzultzeko: **singleQuoted()** funtzioa eguneratzea, puntu batekin dei ahal izateko.

HEDAPEN-FUNTZIOETARAKO SARRERA

Une honetan idatzita dagoenez, **singleQuoted()** funtzioak parametro bakarra du, **original** izenekoa, komatxoaren artean jarriko den katea. Orain, funtzioa eguneratu besterik ez dugu egin behar, ohiko funtzio parametro bat izan beharrean hartzaile bat izan dezan.

What we have: `singleQuoted(title)`



What we want: `title.singleQuoted()`

HEDAPEN-FUNTZIOETARAKO SARRERA

Funtzio bat puntu batekin deitu nahi duzunean, **klaseari funtzioa gehituz** egin daiteke. Hala ere, beti ezingo duzu hori egin. **String** klasea Kotlin-eko liburutegi estandarraren parte da, beraz, ezin duzu bere kodea ireki eta funtzio berria gehitu.

Zorionez, Kotinek klase bat **zabaldu** eta puntu batekin dei daitezkeen funtzioak gehitzeko aukera eskaintzen du. Hauek **luzapen funtzioak** dira.

Ikus dezagun kapitulu honen hasieran idatzi dugun **singleQuoted()** funtzioa.

HEDAPEN-FUNTZIOETARAKO SARRERA

```
fun singleQuoted(original: String) = "'$original'"
```

Aldatu **original** parametroa hartzailea izan dadin, **singleQuoted()** luzapen funtzio bat izan dadin. Erraz egin daiteke:

Lehenik, gehitu nahi duzun hartzaile motaren izena funtzioaren izenaren aurrean eta jarri puntu bat. Kasu honetan, **String** bat izan nahi dugu hartzailea.

Bigarrenik, erabili **this** hartzaileari erreferentzia egiteko funtzioaren gorputzean.

HEDAPEN-FUNTZIOETARAKO SARRERA

```
fun singleQuoted(original: String) = "'$original'"
```

Aldatu **original** parametroa hartzailea izan dadin, **singleQuoted()** luzapen funtzio bat izan dadin. Erraz egin daiteke:

1. Lehenik, gehitu nahi duzun hartzaile motaren izena funtzioaren izenaren aurrean eta jarri puntu bat. Kasu honetan, **String** bat izan nahi dugu hartzailea.
2. Bigarrenik, erabili **this** hartzaileari erreferentzia egiteko funtzioaren gorputzean.

HEDAPEN-FUNTZIOETARAKO SARRERA

Horrela ikusten da **singleQuoted ()** aldaketa horiek egin ondoren

```
fun singleQuoted(original: String) = "'$original'"
```

Kode honetan:

String da **hartzaile mota**. String bat bezala zehazten baduzu, **singleQuoted()** deitu ahal izango diozu edozein **String** objekturi.

this da **hartzailearen parametroa**. **singleQuoted()** deitzen den edozein objekturi

HEDAPEN-FUNTZIOETARAKO SARRERA

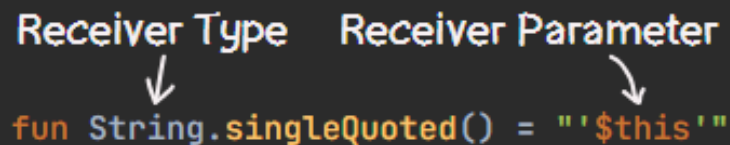
Horrela ikusten da **singleQuoted ()** aldaketa horiek egin ondoren

```
fun singleQuoted(original: String) = "'$original'"
```

Kode honetan:

- **String** da **hartzaile mota**. String bat bezala zehazten baduzu, **singleQuoted()** deitu ahal izango diozu edozein **String** objekturi.
- **this** da **hartzailearen parametroa**. **singleQuoted()** deitzen den edozein objekturi erreferentzia egiten dio, beraz, **title.singleQuoted()** deitzen baduzu, **this** izango da **title** objektua.

HEDAPEN-FUNTZIOETARAKO SARRERA

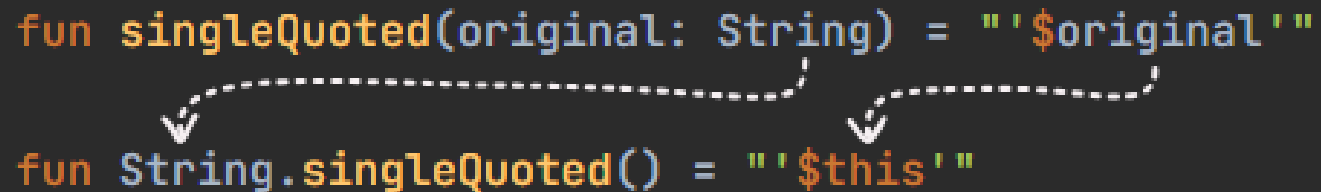


```
Receiver Type  Receiver Parameter  
    ↓           ↓  
fun String.singleQuoted() = ''$this''
```

Funtzio arrunt bat luzapen funtzio bihurtzea erraza da:

1. Jarri parametro motaren izena funtzioaren izenaren aurrean eta jarri puntu bat.
2. Aldatu parametroaren izena **this-ra** funtzioaren barruan.
3. Azkenik, kendu **original** parametroa parentesi artean.

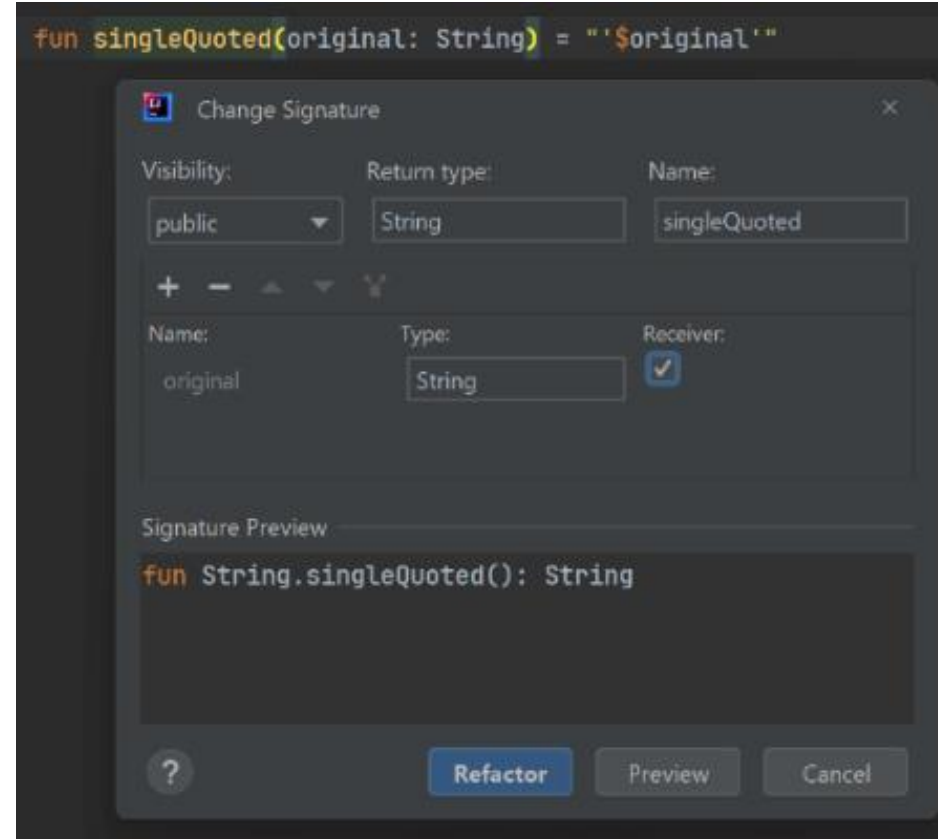
HEDAPEN-FUNTZIOETARAKO SARRERA



```
fun singleQuoted(original: String) = "$original"
fun String.singleQuoted() = "$this"
```

IntelliJ edo Android Studio erabiltzen baduzu, funtzio bat luzapen funtzio bihurtu dezakezu **birsortze tresnekin**. Horretarako, egin eskuin klik funtzioaren izenean, eta aukeratu “Birsortu” eta “Sinadura aldatu”. Hortik aurrera, markatu hartzaile bihurtu nahi duzun parametroa.

HEDAPEN-FUNTZIOETARAKO SARRERA



HEDAPEN-FUNTZIOETARAKO SARRERA

Aldaketa hauekin, funtzio honi dei egin behar diozu **hartzaile bat** erabiliz, honela:

```
val quotedTitle = title.singleQuoted()
```

Eta orain, funtzio-dei hau kate deien erdian txerta dezakezu erraz:

```
val title = "The Robots from Planet X3"
val newTitle = title
    .removePrefix("The ")
    .singleQuoted()
    .uppercase()

// 'ROBOTS FROM PLANET X3'
```

HEDAPEN-FUNTZIOETARAKO SARRERA

Luzapen funtzioak ohikoak dira Kotlin kodean. Kotlin-en liburutegi estandarrak ere luzapen funtzio ugari biltzen ditu. Egia esanda, harritu zaitezke jakitean **removePrefix()** eta **uppercase()** ez direla benetan **String** klasearen kideak: **luzapen funtzioak** dira!

HEDAPEN-FUNTZIOETARAKO SARRERA

Luzapenak modu bikaina dira lehendik dagoen mota bati funtzionaltasun berria emateko, batez ere klasearen kodea editatu ezin duzun kasuetan. Hala ere, kontuan izan luzapenek ezin dutela sarbiderik izan **pribatuak diren** klasearen kideetara. Beraz, luzapen funtzio bati **kide funtzio** bati bezala deitzen bazaio ere, ez du sarbiderik izango **kide funtzioak** duen informazio guztira.

BALIO NULUAK ONARTZEN DITUZTEN HARTZAILE MOTAK

Zer gertatzen da luzapen funtzio bat balio nuluak onartzen dituen objektu batean deitu nahi baduzu?

```
val title: String? = null  
val newTitle = title.singleQuoted()
```

Errore-mezua jasoko duzu.

Hau **segurtasun-deiaren operadorea** erabiliz konpon dezakezu?; horrela, **singleQuoted()** deituko da soilik **title** ez denean nulua.

BALIO NULUAK ONARTZEN DITUZTEN HARTZAILE MOTAK

Hala ere, Kotlin-ek beste aukera bat ere eskaintzen du: **balio nuluak onartzen dituen hartzaile mota bat** duen luzapen funtzio bat sortzea. Adibidez, hartzaile motari nuluak onartzen dituen **String?** gehitu diezaiokezu, honela:

```
fun String?.singleQuoted() =  
    if (this == null) "(no value)" else "'$this'"
```

BALIO NULUAK ONARTZEN DITUZTEN HARTZAILE MOTAK

Funtzio honetan, **this** nulu daiteke. **singleQuoted()** bertsio honi balio nulua ematen bazaio, itzuliko du **(no value)**, bestela, **singleQuoted()** aurreko bertsioak bezala funtzionatuko du.

Luzapen funtzio batek balio nuluak onartzen dituen hartzaile mota badu, ez duzu **segurtasun-deiaren operadorea** erabili behar; ordez, **puntu operadore normal** batekin deitu ahal izango duzu.

BALIO NULUAK ONARTZEN DITUZTEN HARTZAILE MOTAK

Funtzio honetan, **this** nulu daiteke. **singleQuoted()** bertsio honi balio nulua ematen bazaio, itzuliko du **(no value)**, bestela, **singleQuoted()** aurreko bertsioak bezala funtzionatuko du.

Luzapen funtzio batek balio nuluak onartzen dituen hartzaile mota badu, ez duzu **segurtasun-deiaren operadorea** erabili behar; ordez, **puntu operadore normal** batekin deitu ahal izango duzu.

```
val title: String? = null
val newTitle = title.singleQuoted()

println(newTitle) // (no value)
```


BALIO NULUAK ONARTZEN DITUZTEN HARTZAILE MOTAK

Aitzitik, **segurtasun-deiaren operadorea** erabili dezakezu nahi izanez gero, baina kasu horretan funtzioa **hartzailea nulua ez denean soilik** deituko da. Adibidez, hurrengo zerrendaren eta aurrekoaren arteko aldea puntu operadorea segurtasun-deiaren operadorearekin aldatzean datza. Eraitza **newTitle** nulua izatea da, **(no value)** beharrean. Por lo tanto, elija con cuidado entre un operador de puntos y un operador de llamada segura, en función de sus expectativas.

```
val title: String? = null
val newTitle = title?.singleQuoted()

println(newTitle) // null
```

HEDADURAREN PROPIETATEAK

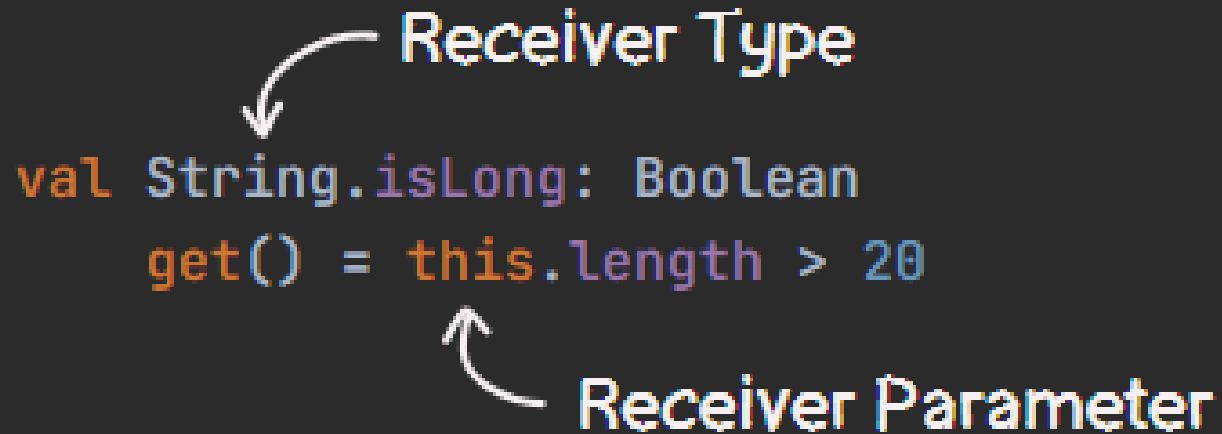
Luzapen funtzioez gain, **luzapen propietateak** ere sor ditzakezu. Hala ere, luzapen propietate bat ezin duzu erabili klase batean balio gehigarriak gordetzeko. Adibidez, ezin duzu identifikazio zenbaki bat gehitu **String** bati. Hala ere, erabilgarriak izan daitezke kalkulu txikiak egiteko.

Sortu dezagun luzapen propietate bat, **String** batek 20 karaktere baino gehiago dituen ala ez esaten digun.

```
val String.isLong: Boolean  
    get() = this.length > 20
```

HEDADURAREN PROPIETATEAK

Luzapen funtzio batekin bezala, luzapen propietate batek **hartzaile mota** zehazten du, eta **hartzaile parametroa this** bezala dago eskuragarri.



```
Receiver Type
val String.isLong: Boolean
    get() = this.length > 20
Receiver Parameter
```

The diagram shows a Kotlin property definition: `val String.isLong: Boolean` followed by an indented `get() = this.length > 20`. A curved arrow points from the text "Receiver Type" to the `String` part of the property declaration. Another curved arrow points from the text "Receiver Parameter" to the `this` keyword inside the `get()` function.

HEDADURAREN PROPIETATEAK

Arestian aipatu bezala, hartzaile isil batean funtzio edo propietate bati deitzen diozunean, ez duzu **this**. jarri beharrik; beraz, propietate hau **this** gabe ere idatz dezakezu:

```
val String.isLong: Boolean  
    get() = length > 20
```

Propietate hau beste edozein propietate bezala erabil dezakezu:

```
val string = "This string is long enough"  
val isItLong = string.isLong
```